UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026

INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema:	APE4: Integración de clases, objetos y métodos
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Primero - B
Alumnos participantes:	Navas Ortiz Jefferson Sebastian Bonifaz Vasconez Francis Josue Punina Moposita Kevin David Prado Falcones Dixon Johao
Asignatura:	Algoritmos y Lógica de Programación
Docente:	Ing. José Rubén Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:
Desarrollar programas utilizando clases, objetos y métodos, lo cual permite disminuir errores y reutilizar el código.

2.2 Instrucciones

- La tarea se realiza de forma individual
- Leer cada ejercicio planteado en el aula virtual y codificarlo en Java
- Comprimir y subir al aula virtual el proyecto desarrollado
- Realizar capturas de las clases y métodos en un PDF con el formato de Guías APE

2.3 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC.
- Computador o laptop.
- Plataforma educativa de la UTA.
- Internet para consultas (fuentes confiables).

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☐Simuladores y laboratorios virtuales
☐Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☒Inteligencia Artificial
Otros (Especifique): _____

2.4 Actividades por desarrollar

Desarrollar la APE 4 en tres fases: Fase 1, Introducción a la POO en Java (Notas del curso, Inventario de una tienda, Veterinaria, Cuenta de ahorro y Biblioteca); Fase 2, Métodos en Java (identificación de tipos de retorno); y Fase 3, Integración de clases, objetos y métodos (Libro y Autor, Rectángulo, Producto y Venta, Transferencias y Biblioteca).

Repositorio en GitHub: https://github.com/dixonp10/APE-4_CLASES_OBJETOS_METODOS

2.5 Resultados obtenidos

FASE 1: Introducción a la POO en Java

Ejercicio 1: Notas del curso

Enunciado: Crear la clase Estudiante con los atributos nombre, nota1, nota2, nota3 y promedio. En main, crear 5 instancias, calcular el promedio de cada una e imprimirlas una debajo de otra con el formato nombre : promedio, agregando Aprobado (7 o más), Supletorio (de 5 a 6.99) o Reprobado (menos de 5). Al final, imprimir cuántos estudiantes aprobaron.

Código:

Estudiante.java

```
package ejercicio1;

public class Estudiante {
    String nombre;
    double nota1;
    double nota2;
    double nota3;
    double promedio;

    public Estudiante(String nombre, double nota1, double nota2, double nota3) {
        this.nombre = nombre;
        this.nota1 = nota1;
        this.nota2 = nota2;
        this.nota3 = nota3;
    }

    public void calcularPromedio() {
        promedio = (nota1 + nota2 + nota3) / 3;
    }

    public String estado() {
        if (promedio >= 7) {
            return "Aprobado";
        } else if (promedio >= 5) {
            return "Supletorio";
        } else {
            return "Reprobado";
        }
    }
}
```

Main.java

```
package ejercicio1;

import java.util.Locale;

public class Main {
    public static void main(String[] args) {
        Estudiante e1 = new Estudiante("Ana", 9.0, 8.5, 9.5);
        Estudiante e2 = new Estudiante("Luis", 7.0, 6.5, 7.5);
        Estudiante e3 = new Estudiante("Maria", 5.0, 6.0, 6.5);
        Estudiante e4 = new Estudiante("Pedro", 4.0, 3.5, 5.0);
        Estudiante e5 = new Estudiante("Sofia", 8.0, 7.5, 9.0);

        Estudiante[] estudiantes = {e1, e2, e3, e4, e5};
        int aprobados = 0;

        System.out.println("=== NOTAS DEL CURSO ===");
        for (Estudiante e : estudiantes) {
```

```
            e.calcularPromedio();
            System.out.println(e.nombre + " : "
                    + String.format(Locale.US, "%.2f", e.promedio)
                    + " " + e.estado());
            if (e.promedio >= 7) {
                aprobados++;
            }
        }

        System.out.println("Estudiantes aprobados: " + aprobados);
    }
}
```

Prueba de ejecución:

```
=== NOTAS DEL CURSO ===
Ana : 9.00 Aprobado
Luis : 7.00 Aprobado
Maria : 5.83 Supletorio
Pedro : 4.17 Reprobado
Sofia : 8.17 Aprobado
Estudiantes aprobados: 3
```

Ejercicio 2: Inventario de una tienda

Enunciado: Crear la clase Producto con nombre, precio y stock. Crear 4 productos e imprimirlos con el formato nombre : precio. Simular la venta de 3 unidades de dos de ellos, mostrar “Reponer” junto a cada producto que quede con menos de 5 unidades e imprimir el valor total del inventario (suma de precio por stock).

Código:

Producto.java

```
package ejercicio2;

public class Producto {
    String nombre;
    double precio;
    int stock;

    public Producto(String nombre, double precio, int stock) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public void vender(int cantidad) {
        stock = stock - cantidad;
    }
}
```

Main.java

```
package ejercicio2;

import java.util.Locale;

public class Main {
    public static void main(String[] args) {
        Producto p1 = new Producto("Laptop", 850.00, 10);
        Producto p2 = new Producto("Mouse", 15.50, 20);
        Producto p3 = new Producto("Teclado", 32.00, 6);
```

```
        Producto p4 = new Producto("Monitor", 210.00, 7);

        Producto[] productos = {p1, p2, p3, p4};

        System.out.println("=== INVENTARIO INICIAL ===");
        for (Producto p : productos) {
            System.out.println(p.nombre + " : "
                    + String.format(Locale.US, "%.2f", p.precio));
        }

        // Simulacion de ventas: 3 unidades de dos productos
        p3.vender(3);
        p4.vender(3);
        System.out.println();
        System.out.println("Venta realizada: 3 unidades de Teclado y 3 de Monitor");

        System.out.println();
        System.out.println("=== INVENTARIO DESPUES DE LAS VENTAS ===");
        double total = 0;
        for (Producto p : productos) {
            String linea = p.nombre + " : "
                    + String.format(Locale.US, "%.2f", p.precio)
                    + " (stock: " + p.stock + ")";
            if (p.stock < 5) {
                linea += " Reponer";
            }
            System.out.println(linea);
            total += p.precio * p.stock;
        }

        System.out.println("Valor total del inventario: "
                + String.format(Locale.US, "%.2f", total));
    }
}
```

Prueba de ejecución:

```
=== INVENTARIO INICIAL ===
Laptop : 850.00
Mouse : 15.50
Teclado : 32.00
Monitor : 210.00

Venta realizada: 3 unidades de Teclado y 3 de Monitor

=== INVENTARIO DESPUES DE LAS VENTAS ===
Laptop : 850.00 (stock: 10)
Mouse : 15.50 (stock: 20)
Teclado : 32.00 (stock: 3) Reponer
Monitor : 210.00 (stock: 4) Reponer
Valor total del inventario: 9746.00
```

Ejercicio 3: Veterinaria

Enunciado: Crear la clase Mascota con nombre, especie, edad y peso. Crear 3 mascotas e imprimirlas con el formato nombre : especie : peso, mostrando “Revisar peso” si pesan más de 20 kg. Luego escribir Mascota m4 = m1;, sumar 1 a m4.edad, imprimir m1.edad y explicar en un comentario por qué cambió.

Código:

Mascota.java

```
package ejercicio3;

public class Mascota {
    String nombre;
    String especie;
    int edad;
    double peso;

    public Mascota(String nombre, String especie, int edad, double peso) {
        this.nombre = nombre;
        this.especie = especie;
        this.edad = edad;
        this.peso = peso;
    }
}
```

Main.java

```
package ejercicio3;

import java.util.Locale;

public class Main {
    public static void main(String[] args) {
        Mascota m1 = new Mascota("Firulais", "Perro", 5, 25.5);
        Mascota m2 = new Mascota("Misu", "Gato", 3, 4.2);
        Mascota m3 = new Mascota("Rocky", "Perro", 7, 32.0);

        Mascota[] mascotas = {m1, m2, m3};

        System.out.println("=== VETERINARIA ===");
        for (Mascota m : mascotas) {
            String linea = m.nombre + " : " + m.especie + " : "
                    + String.format(Locale.US, "%.1f", m.peso) + " kg";
            if (m.peso > 20) {
                linea += " Revisar peso";
            }
            System.out.println(linea);
        }

        System.out.println();
        System.out.println("=== REFERENCIAS ===");
        Mascota m4 = m1;
        m4.edad = m4.edad + 1;   // m4.edad += 1
        System.out.println("Edad de m1: " + m1.edad);

        /*
         * Explicacion: en Java las variables de objetos guardan una REFERENCIA
         * (la direccion en memoria del objeto), no una copia del objeto.
         * Al escribir "Mascota m4 = m1;" no se crea una mascota nueva: m4 y m1
         * apuntan al MISMO objeto. Por eso, al sumar 1 a m4.edad, el cambio se
         * ve tambien en m1.edad (paso de 5 a 6).
         */
    }
}
```

Prueba de ejecución:

```
=== VETERINARIA ===
```

```
Firulais : Perro : 25.5 kg Revisar peso
Misu : Gato : 4.2 kg
Rocky : Perro : 32.0 kg Revisar peso

=== REFERENCIAS ===
Edad de m1: 6
```

Ejercicio 4: Cuenta de ahorro

Enunciado: Crear la clase Cuenta con titular y saldo. Crear 2 cuentas, una con 200 y otra con 350. Con un while, depositar 50 por mes en cada cuenta hasta que llegue a 500, imprimiendo cada mes con el formato titular : saldo, y al final cuántos meses necesitó cada una (6 y 3).

Código:

Cuenta.java

```
package ejercicio4;

public class Cuenta {
    String titular;
    double saldo;

    public Cuenta(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }
}
```

Main.java

```
package ejercicio4;

public class Main {
    public static void main(String[] args) {
        Cuenta c1 = new Cuenta("Carlos", 200);
        Cuenta c2 = new Cuenta("Daniela", 350);

        System.out.println("=== CUENTA DE AHORRO: " + c1.titular + " ===");
        int meses1 = 0;
        while (c1.saldo < 500) {
            c1.saldo += 50;
            meses1++;
            System.out.println("Mes " + meses1 + " - " + c1.titular
                    + " : " + (int) c1.saldo);
        }

        System.out.println();
        System.out.println("=== CUENTA DE AHORRO: " + c2.titular + " ===");
        int meses2 = 0;
        while (c2.saldo < 500) {
            c2.saldo += 50;
            meses2++;
            System.out.println("Mes " + meses2 + " - " + c2.titular
                    + " : " + (int) c2.saldo);
        }

        System.out.println();
        System.out.println(c1.titular + " necesito " + meses1 + " meses");
        System.out.println(c2.titular + " necesito " + meses2 + " meses");
    }
}
```

Prueba de ejecución:

```
=== CUENTA DE AHORRO: Carlos ===
Mes 1 - Carlos : 250
Mes 2 - Carlos : 300
Mes 3 - Carlos : 350
Mes 4 - Carlos : 400
Mes 5 - Carlos : 450
Mes 6 - Carlos : 500

=== CUENTA DE AHORRO: Daniela ===
Mes 1 - Daniela : 400
Mes 2 - Daniela : 450
Mes 3 - Daniela : 500

Carlos necesito 6 meses
Daniela necesito 3 meses
```

Ejercicio 5: Biblioteca

Enunciado: Crear las clases Libro (titulo, autor, disponible) y Socio (nombre, librosPrestados). Crear 4 libros y 2 socios, registrar que el primer socio presta 2 libros y el segundo presta 1 (disponible = false y aumentar librosPrestados), y al final imprimir cada libro con el formato titulo : Disponible o titulo : Prestado, cada socio con el formato nombre : librosPrestados y cuántos libros quedan disponibles.

Código:

Libro.java

```
package ejercicio5;

public class Libro {
    String titulo;
    String autor;
    boolean disponible;

    public Libro(String titulo, String autor) {
        this.titulo = titulo;
        this.autor = autor;
        this.disponible = true;
    }
}
```

Socio.java

```
package ejercicio5;

public class Socio {
    String nombre;
    int librosPrestados;

    public Socio(String nombre) {
        this.nombre = nombre;
        this.librosPrestados = 0;
    }

    public void prestar(Libro libro) {
        libro.disponible = false;
        librosPrestados++;
    }
}
```

Main.java

```
package ejercicio5;

public class Main {
    public static void main(String[] args) {
        Libro l1 = new Libro("Cien anos de soledad", "Gabriel Garcia Marquez");
        Libro l2 = new Libro("Don Quijote de la Mancha", "Miguel de Cervantes");
        Libro l3 = new Libro("Huasipungo", "Jorge Icaza");
        Libro l4 = new Libro("El principito", "Antoine de Saint-Exupery");

        Socio s1 = new Socio("Juan Perez");
        Socio s2 = new Socio("Laura Mena");

        Libro[] libros = {l1, l2, l3, l4};
        Socio[] socios = {s1, s2};

        // El primer socio presta 2 libros y el segundo presta 1
        s1.prestar(l1);
        s1.prestar(l2);
        s2.prestar(l3);

        System.out.println("=== LIBROS ===");
        int disponibles = 0;
        for (Libro l : libros) {
            if (l.disponible) {
                System.out.println(l.titulo + " : Disponible");
                disponibles++;
            } else {
                System.out.println(l.titulo + " : Prestado");
            }
        }

        System.out.println();
        System.out.println("=== SOCIOS ===");
        for (Socio s : socios) {
            System.out.println(s.nombre + " : " + s.librosPrestados);
        }

        System.out.println();
        System.out.println("Libros disponibles: " + disponibles);
    }
}
```

Prueba de ejecución:

```
=== LIBROS ===
Cien anos de soledad : Prestado
Don Quijote de la Mancha : Prestado
Huasipungo : Prestado
El principito : Disponible

=== SOCIOS ===
Juan Perez : 2
Laura Mena : 1

Libros disponibles: 1
```

FASE 2: Métodos en Java

Ejercicio 1: Identificación de tipos de retorno

Enunciado: El siguiente método debe calcular el área de un rectángulo, pero genera un error de compilación. Identificar por qué falla y proponer las correcciones.

Código original (genera error de compilación):

Ejercicio1Original.java

```
class Ejercicio1Original {

    public int calcularArea(double base, double altura) {
        double area = base * altura;
        System.out.println("El área es: " + area);
    }
}
```

Salida de la compilación:

```
Ejercicio1Original.java:6: error: missing return statement
    }
    ^
1 error
```

¿Por qué falla?

El método declara que devuelve un int, pero nunca ejecuta una sentencia return. En Java, todo método con tipo de retorno distinto de void debe devolver un valor en todos sus caminos de ejecución. Además, area es un double y no se puede devolver como int sin un casting explícito, porque se perdería la parte decimal.

Correcciones posibles:

A) Cambiar el tipo de retorno a double y agregar return area; (la mejor opción, porque el área puede tener decimales).

B) Cambiar el tipo de retorno a void si solo se desea imprimir el resultado.

Código:

Ejercicio1.java

```
package ejercicio1;

/**
 * EJERCICIOS APE
 *
 * Ejercicio 1: Identificación de tipos de retorno
 */
class Ejercicio1 {

    // Corrección A: devuelve el área como double
    public double calcularArea(double base, double altura) {
        double area = base * altura;
        System.out.println("El área es: " + area);
        return area; // <- sentencia que faltaba
    }

    // Corrección B: solo imprime, por lo tanto es void
    public void calcularAreaVoid(double base, double altura) {
        double area = base * altura;
        System.out.println("El área es: " + area);
    }
}
```

Main.java

```
package ejercicio1;

/**
 * Prueba de las dos correcciones del Ejercicio 1.
 */
public class Main {
    public static void main(String[] args) {
        Ejercicio1 ejercicio = new Ejercicio1();

        System.out.println("=== CORRECCIÓN A: calcularArea (devuelve double) ===");
        double resultado = ejercicio.calcularArea(5.0, 3.0);
        System.out.println("Valor devuelto por el método: " + resultado);
        System.out.println("Doble del área (usando el valor devuelto): " + (resultado * 2));

        System.out.println();
        System.out.println("=== CORRECCIÓN B: calcularAreaVoid (void) ===");
        ejercicio.calcularAreaVoid(4.5, 2.0);
    }
}
```

Prueba de ejecución:

```
=== CORRECCIÓN A: calcularArea (devuelve double) ===
El área es: 15.0
Valor devuelto por el método: 15.0
Doble del área (usando el valor devuelto): 30.0

=== CORRECCIÓN B: calcularAreaVoid (void) ===
El área es: 9.0
```

FASE 3: Integración de clases, objetos y métodos

Ejercicio 1: Libro y Autor

Enunciado: Crear la clase Autor (nombre, nacionalidad) y la clase Libro (titulo, anio, autor). Libro debe tener un atributo de tipo Autor y un método mostrarInformacion() que imprima el título, el año y el nombre del autor. En main, crear 2 libros.

Código:

Autor.java

```
package ejercicio1;

public class Autor {
    private String nombre;
    private String nacionalidad;

    public Autor(String nombre, String nacionalidad) {
        this.nombre = nombre;
        this.nacionalidad = nacionalidad;
    }

    public String getNombre() {
        return nombre;
    }

    public String getNacionalidad() {
        return nacionalidad;
    }
}
```

Libro.java

```
package ejercicio1;

public class Libro {
    private String titulo;
    private int anio;
    private Autor autor; // objeto como atributo

    public Libro(String titulo, int anio, Autor autor) {
        this.titulo = titulo;
        this.anio = anio;
        this.autor = autor;
    }

    // Imprime el titulo, el anio y el nombre del autor
    public void mostrarInformacion() {
        System.out.println("Título: " + titulo);
        System.out.println("Año: " + anio);
        System.out.println("Autor: " + autor.getNombre());
    }
}
```

Main.java

```
package ejercicio1;

public class Main {
    public static void main(String[] args) {
        Autor autor1 = new Autor("Gabriel García Márquez", "Colombiana");
        Autor autor2 = new Autor("Jorge Icaza", "Ecuatoriana");

        Libro libro1 = new Libro("Cien años de soledad", 1967, autor1);
        Libro libro2 = new Libro("Huasipungo", 1934, autor2);

        System.out.println("=== LIBRO 1 ===");
        libro1.mostrarInformacion();
        System.out.println();
        System.out.println("=== LIBRO 2 ===");
        libro2.mostrarInformacion();
    }
}
```

Prueba de ejecución:

```
=== LIBRO 1 ===
Título: Cien años de soledad
Año: 1967
Autor: Gabriel García Márquez

=== LIBRO 2 ===
Título: Huasipungo
Año: 1934
Autor: Jorge Icaza
```

Ejercicio 2: Rectángulo

Enunciado: Crear la clase Rectangulo (base, altura) con los métodos calcularArea(), calcularPerimetro(), esCuadrado(), obtenerDescripcion() y mostrar(), usando el tipo de retorno correcto en cada firma.

Código:

Rectangulo.java

```
package ejercicio2;

public class Rectangulo {
    private double base;
    private double altura;

    public Rectangulo(double base, double altura) {
        this.base = base;
        this.altura = altura;
    }

    // Devuelve el area: base * altura
    public double calcularArea() {
        return base * altura;
    }

    // Devuelve el perimetro: 2 * (base + altura)
    public double calcularPerimetro() {
        return 2 * (base + altura);
    }

    // Es cuadrado cuando la base y la altura son iguales
    public boolean esCuadrado() {
        return base == altura;
    }

    // Devuelve una descripcion, por ejemplo "Rectángulo de 4.0 x 6.0"
    public String obtenerDescripcion() {
        return "Rectángulo de " + base + " x " + altura;
    }

    // Imprime todos los resultados
    public void mostrar() {
        System.out.println(obtenerDescripcion());
        System.out.println("Área: " + calcularArea());
        System.out.println("Perímetro: " + calcularPerimetro());
        System.out.println("¿Es cuadrado?: " + (esCuadrado() ? "Sí" : "No"));
    }
}
```

Main.java

```
package ejercicio2;

public class Main {
    public static void main(String[] args) {
        Rectangulo r1 = new Rectangulo(4.0, 6.0);
        Rectangulo r2 = new Rectangulo(5.0, 5.0);

        System.out.println("=== RECTÁNGULO 1 ===");
        r1.mostrar();
        System.out.println();
        System.out.println("=== RECTÁNGULO 2 ===");
        r2.mostrar();
    }
}
```

Prueba de ejecución:

```
=== RECTÁNGULO 1 ===
Rectángulo de 4.0 x 6.0
Área: 24.0
Perímetro: 20.0
¿Es cuadrado?: No

=== RECTÁNGULO 2 ===
Rectángulo de 5.0 x 5.0
Área: 25.0
Perímetro: 20.0
¿Es cuadrado?: Sí
```

Ejercicio 3: Producto y Venta

Enunciado: Crear la clase Producto (nombre, precio, stock) con reducirStock(int cantidad), que solo actúa si hay stock suficiente (si no, imprime “Stock insuficiente.”), y mostrarProducto(). Crear la clase Venta (cantidad) con aplicar(Producto producto), que reduzca el stock y muestre el total a pagar.

Código:

Producto.java

```
package ejercicio3;

public class Producto {
    private String nombre;
    private double precio;
    private int stock;

    public Producto(String nombre, double precio, int stock) {
        this.nombre = nombre;
        this.precio = precio;
        this.stock = stock;
    }

    public String getNombre() {
        return nombre;
    }

    public double getPrecio() {
        return precio;
    }

    public int getStock() {
        return stock;
    }

    // Solo reduce el stock si hay unidades suficientes.
    // Devuelve true si pudo reducirlo y false si no.
    public boolean reducirStock(int cantidad) {
        if (cantidad > 0 && cantidad <= stock) {
            stock = stock - cantidad;
            return true;
        }
        System.out.println("Stock insuficiente.");
        return false;
    }

    public void mostrarProducto() {
```

```
        System.out.println("Producto: " + nombre + " | Precio: $" + String.format("%.2f", precio)
                + " | Stock: " + stock);
    }
}
```

Venta.java

```
package ejercicio3;

public class Venta {
    private int cantidad;

    public Venta(int cantidad) {
        this.cantidad = cantidad;
    }

    // Recibe un Producto como parametro, reduce su stock y muestra el total a pagar
    public void aplicar(Producto producto) {
        System.out.println("Venta de " + cantidad + " unidad(es) de " + producto.getNombre());
        if (producto.reducirStock(cantidad)) {
            double total = cantidad * producto.getPrecio();
            System.out.println("Total a pagar: $" + String.format("%.2f", total));
        }
    }
}
```

Main.java

```
package ejercicio3;

public class Main {
    public static void main(String[] args) {
        Producto cuaderno = new Producto("Cuaderno", 2.50, 10);

        System.out.println("=== ANTES DE LAS VENTAS ===");
        cuaderno.mostrarProducto();

        System.out.println();
        System.out.println("=== VENTA 1 (hay stock) ===");
        Venta venta1 = new Venta(4);
        venta1.aplicar(cuaderno);
        cuaderno.mostrarProducto();

        System.out.println();
        System.out.println("=== VENTA 2 (stock insuficiente) ===");
        Venta venta2 = new Venta(8);
        venta2.aplicar(cuaderno);
        cuaderno.mostrarProducto();
    }
}
```

Prueba de ejecución:

```
=== ANTES DE LAS VENTAS ===
Producto: Cuaderno | Precio: $2.50 | Stock: 10

=== VENTA 1 (hay stock) ===
Venta de 4 unidad(es) de Cuaderno
Total a pagar: $10.00
Producto: Cuaderno | Precio: $2.50 | Stock: 6

=== VENTA 2 (stock insuficiente) ===
Venta de 8 unidad(es) de Cuaderno
```

```
Stock insuficiente.
Producto: Cuaderno | Precio: $2.50 | Stock: 6
```

Ejercicio 4: Transferencias

Enunciado: Usar la clase Cuenta (titular, saldo) y agregar el método transferir(Cuenta destino, double monto), que retire el monto de la cuenta actual y lo deposite en la cuenta destino solo si hay fondos. En main, crear 2 cuentas, hacer una transferencia válida y otra que falle.

Código:

Cuenta.java

```
package ejercicio4;

public class Cuenta {
    private String titular;
    private double saldo;

    public Cuenta(String titular, double saldo) {
        this.titular = titular;
        this.saldo = saldo;
    }

    public String getTitular() {
        return titular;
    }

    public double getSaldo() {
        return saldo;
    }

    public void depositar(double monto) {
        saldo = saldo + monto;
    }

    // Retira el monto solo si es positivo y hay fondos suficientes
    public boolean retirar(double monto) {
        if (monto > 0 && monto <= saldo) {
            saldo = saldo - monto;
            return true;
        }
        return false;
    }

    // Retira de esta cuenta y deposita en la cuenta destino (objeto como parametro)
    public void transferir(Cuenta destino, double monto) {
        if (retirar(monto)) {
            destino.depositar(monto);
            System.out.println("Transferencia exitosa de $" + String.format("%.2f", monto)
                    + " de " + titular + " a " + destino.getTitular());
        } else {
            System.out.println("Transferencia fallida: fondos insuficientes o monto inválido ($"
                    + String.format("%.2f", monto) + ").");
        }
    }

    public void mostrarSaldo() {
        System.out.println(titular + " : $" + String.format("%.2f", saldo));
    }
}
```

Main.java

```
package ejercicio4;

public class Main {
    public static void main(String[] args) {
        Cuenta cuenta1 = new Cuenta("Carlos", 200);
        Cuenta cuenta2 = new Cuenta("Daniela", 350);

        System.out.println("=== SALDOS INICIALES ===");
        cuenta1.mostrarSaldo();
        cuenta2.mostrarSaldo();

        System.out.println();
        System.out.println("=== TRANSFERENCIA VÁLIDA ===");
        cuenta1.transferir(cuenta2, 150);
        cuenta1.mostrarSaldo();
        cuenta2.mostrarSaldo();

        System.out.println();
        System.out.println("=== TRANSFERENCIA QUE FALLA ===");
        cuenta1.transferir(cuenta2, 500);
        cuenta1.mostrarSaldo();
        cuenta2.mostrarSaldo();
    }
}
```

Prueba de ejecución:

```
=== SALDOS INICIALES ===
Carlos : $200.00
Daniela : $350.00

=== TRANSFERENCIA VÁLIDA ===
Transferencia exitosa de $150.00 de Carlos a Daniela
Carlos : $50.00
Daniela : $500.00

=== TRANSFERENCIA QUE FALLA ===
Transferencia fallida: fondos insuficientes o monto inválido ($500.00).
Carlos : $50.00
Daniela : $500.00
```

Ejercicio 5: Biblioteca

Enunciado: Crear tres clases que colaboren entre sí: Usuario, Libro y Prestamo. Prestamo tiene atributos de tipo Usuario y Libro, y su método registrar() intenta prestar el libro y muestra un mensaje según el resultado. En main, hacer que dos usuarios intenten pedir el mismo libro.

Código:

Usuario.java

```
package ejercicio5;

public class Usuario {
    private String nombre;
    private String cedula;

    public Usuario(String nombre, String cedula) {
        this.nombre = nombre;
        this.cedula = cedula;
```

```
    }

    public String getNombre() {
        return nombre;
    }

    public String getCedula() {
        return cedula;
    }
}
```

Libro.java

```
package ejercicio5;

public class Libro {
    private String titulo;
    private boolean disponible;

    public Libro(String titulo) {
        this.titulo = titulo;
        this.disponible = true;
    }

    public String getTitulo() {
        return titulo;
    }

    public boolean isDisponible() {
        return disponible;
    }

    // Presta el libro solo si esta disponible
    public boolean prestar() {
        if (disponible) {
            disponible = false;
            return true;
        }
        return false;
    }

    public void devolver() {
        disponible = true;
    }
}
```

Prestamo.java

```
package ejercicio5;

public class Prestamo {
    private Usuario usuario; // objeto como atributo
    private Libro libro;     // objeto como atributo

    public Prestamo(Usuario usuario, Libro libro) {
        this.usuario = usuario;
        this.libro = libro;
    }

    // Intenta prestar el libro y muestra un mensaje segun el resultado
    public void registrar() {
        if (libro.prestar()) {
```

```
            System.out.println("Préstamo registrado: \"" + libro.getTitulo() + "\" fue prestado a "
                    + usuario.getNombre() + " (C.I. " + usuario.getCedula() + ").");
        } else {
            System.out.println("No se pudo registrar: \"" + libro.getTitulo() + "\" no está disponible para "
                    + usuario.getNombre() + " (C.I. " + usuario.getCedula() + ").");
        }
    }
}
```

Main.java

```
package ejercicio5;

public class Main {
    public static void main(String[] args) {
        Libro libro = new Libro("El principito");
        Usuario usuario1 = new Usuario("Juan Pérez", "1805123456");
        Usuario usuario2 = new Usuario("Laura Mena", "1803987654");

        Prestamo prestamo1 = new Prestamo(usuario1, libro);
        Prestamo prestamo2 = new Prestamo(usuario2, libro);

        System.out.println("=== DOS USUARIOS PIDEN EL MISMO LIBRO ===");
        prestamo1.registrar(); // el libro esta disponible
        prestamo2.registrar(); // el libro ya fue prestado

        System.out.println();
        System.out.println("=== DEVOLUCIÓN Y NUEVO INTENTO ===");
        libro.devolver();
        System.out.println("Libro devuelto: \"" + libro.getTitulo() + "\".");
        prestamo2.registrar(); // ahora si esta disponible
    }
}
```

Prueba de ejecución:

```
=== DOS USUARIOS PIDEN EL MISMO LIBRO ===
Préstamo registrado: "El principito" fue prestado a Juan Pérez (C.I. 1805123456).
No se pudo registrar: "El principito" no está disponible para Laura Mena (C.I. 1803987654).

=== DEVOLUCIÓN Y NUEVO INTENTO ===
Libro devuelto: "El principito".
Préstamo registrado: "El principito" fue prestado a Laura Mena (C.I. 1803987654).
```

2.6 Habilidades blandas empleadas en la práctica

☐ Liderazgo
☒ Trabajo en equipo
☐ Comunicación asertiva
☐ La empatía
☐ Pensamiento crítico
☐ Flexibilidad
☐ La resolución de conflictos
☐ Adaptabilidad
☒ Responsabilidad

2.7 Conclusiones

- El uso de la programación orientada a objetos permite disminuir errores en el código y reutilizar código mediante clases y métodos.
- Los objetos pueden ser atributos o parámetros de otras clases, lo que permite que colaboren entre sí (Libro y Autor, Venta y Producto, Cuenta y Cuenta, Prestamo con Usuario y Libro) y explica por qué dos referencias pueden apuntar al mismo objeto.
- Todo método con tipo de retorno distinto de void debe devolver un valor en todos sus caminos; elegir bien el tipo de retorno (boolean, double, String o void) evita errores de compilación y permite validar condiciones como stock, fondos y disponibilidad.

2.8 Referencias bibliográficas

[1] M. I. Barquilla Galeano, Java programación: teoría y ejemplos, 1.ª ed. Madrid, España: RA-MA Editorial, 2025.
[2] F. Blasco, Programación orientada a objetos en Java, 1.ª ed. Bogotá, Colombia: Ediciones de la U, 2019.

2.9 Anexos

Repositorio en GitHub: https://github.com/dixonp10/APE-4_CLASES_OBJETOS_METODOS

Estructura del repositorio:

```
APE-4_CLASES_OBJETOS_METODOS/
├── Fase1/
│   ├── src/
│   ├── pruebas/
│   └── README.md
├── Fase2/
│   ├── src/
│   ├── pruebas/
│   └── README.md
├── Fase3/
│   ├── src/
│   ├── pruebas/
│   └── README.md
├── README.md      (general)
└── Informe.pdf
```